Playwright Automation Field Guide

Everybody has a cheat sheet, we have a Field Guide.

Lists of commands. Snippets without context. Pages of syntax that does not always make sense.

This is not a cheat sheet. It is a field guide that adds context to those snippets and syntax.

This Playwright Field Guide is designed as a practical, structured reference for modern test automation. It doesn't just show you what commands exist — it shows you when to use them, why they matter, and how they work together. Each section builds on the previous one, guiding you through navigation, locating elements, synchronization, actions, assertions, and beyond.

This guide is meant to be printed, shared, and used in real projects — a quick-glance tactical reference for engineers who want stable, maintainable, and modern Playwright tests.

Stable tests are not written by accident. Practice makes perfect.

Browser & Context

Authenticate easy with different users and context.

Working with context

browser.newContext()	Create a new isolated browser context (like a fresh browser profile).
context.newPage()	Open a new page (tab) inside the current browser context.
page.close();	Close the current page (tab).
context.close();	Close the browser context and all pages within it.

Working with cookies

context.cookies();	Get all cookies for the current browser context.
context.addCookies();	Add cookies to the current browser context.
context.clearCookies();	Remove all cookies from the current browser context.

Do

Context

Use context when running tests in isolation.
Simulating multiple users.
Creating clean sessions without shared cookies or storage.
Close context when not used

Cookies

Cookie control is useful for:

- Pre-authenticated sessions
- Testing login states
- Simulating returning users
- Clearing session data between tests

Authenticate once, then reuse it.

Using Stored Authentication (Storage State)

browser.newContext({ storageState: 'auth.json' });	Create a browser context using saved authentication state.
await page.context().storageState({ path: 'auth.json' });	Save the login session.
browser.newContext({ storageState: 'auth.json' });	Load login session.

This is commonly used to:

- Speed up test suites
- Skip login steps
- Share authentication across multiple tests

Don't

const context = await browser.newContext();
Login for every test, use context.
Manipulate cookies unless necessary.
Leave browser context open.

Multiple users

const **adminContext** = await browser.newContext();
const **userContext** = await browser.newContext();

const **adminPage** = await adminContext.newPage();
const **userPage** = await userContext.newPage();

Both the admin and the user will have separate cookies, sessions and local storage. Create multiple users when testing social media, chat, or other multi-user applications.

Next: Frames & Windows — Because not everything lives in the main page.

Frames & Windows

Frames are used when content is embedded inside an <iframe>

Visibility – UI

page.frame(name);	Get a specific frame by name or URL.
page.frames();	Get a list of all frames on the page.
frame.locator();	Locate elements inside a specific frame.
page.waitForEvent('popup');	Wait for a new browser window or tab to open.
context.pages();	Get all open pages in the current browser context.
page.frameLocator();	Locate elements inside an iframe without manually selecting the frame.

How to use it:

```
const frame = page.frame('payment-frame');
await frame.locator('#card-number').fill('4111 1111 1111 1111');
```

When clicking a link that opens a new window: const popup = await page.waitForEvent('popup');

Frames are often used in:

payment providers (Stripe, PayPal)
embedded dashboards
authentication widgets
ads or external content
legacy web applications
Understanding frames helps prevent false locator failures.

To Know

Frames embed content inside a page, while popups open new pages or windows.
Frames require frame access, popups require event handling.
Iframes often belong to third-party systems (payments, ads, widgets).

Don't

Don't interact with iframe elements from the main page.
Don't assume the popup opened instantly.
Don't use multiple pages for frame interaction.
Don't ignore multiple frames on a page

Do

Use frameLocator() when possible.
Always capture the popup when clicking elements that open a new window.
Use frame name, URL, or selector to ensure you target the correct iframe.
Treat popups like normal pages
Always confirm you are interacting with the correct frame.
Test only what your application controls.

Next: Navigators — Because every test starts by opening or moving between pages.

Navigators

Navigators control how your test moves through the application. Every test begins with movement — but reliable tests validate where they land.

Page Movement

page.goto(URL)	`await page.goto('https://playground.testduo.co.za');`
page.goBack()	`await page.goBack();`
page.goForward()	`await page.goForward();`
page.reload()	`await page.reload();`

Page movement is used to traverse between pages, and to reload

Page Information

page.url()	`const currentUrl = page.url();`
page.title()	`const pageTitle = await page.title();`

Page information is used to get information from the current page.

Do

- Validate URL after navigation.
- Use Promise.all() when clicking triggers navigation.
- Use web-first assertions instead of manual waits.
- Think in states, not clicks.

Don't

- Assume navigation succeeded.
- Use waitForTimeout() after clicking a link.
- Rely only on goto() without validation.
- Forget that SPAs may change URL without full reload.

Navigate

Navigate to a page and wait for an element as best practice.

```
await Promise.all([
  page.waitForURL('**/dashboard'),
  page.getByRole('button', { name: 'Login' }).click(),
]);
```

Then Validate

```
await expect(page).toHaveURL('/dashboard');
await expect(page).toHaveTitle('Dashboard');
```

Refresh and Revalidate

```
await page.reload();
await expect(page.getByText('Order placed')).toBeVisible();
```

Not all navigation reloads the page.

Traditional apps → full page load

SPAs → URL changes, DOM updates, no reload

This is why waitForURL() matters more than waitForLoadState() in many modern apps.

Next: Locators — Because tests can't interact with elements they can't find.

Locators

Locators define how your test sees the page. Good locators create stable tests. Bad locators create hard waits.

Accessible First (Preferred)

page.getByRole()	await page.getByRole('button', { name: 'Login' }).click();
page.getByText()	await expect(page.getByText('Invalid email or password')).toBeVisible();
page.getByLabel()	await page.getByLabel('Email').fill('');

Attribute-Based

page.getByPlaceholder()	await page.getByPlaceholder('Enter your password').fill('P@ssw0rd!');
page.getByAltText()	await page.getByAltText('Product image').click();
page.getByTitle()	await page.getByTitle('View details').click();
page.getByTestId()	await page.getByTestId('checkout-submit').click();

CSS / Advanced

page.locator()	await page.locator('#cart-count').textContent();

First / Nth / Last

```
await page.getByRole('button').first().click();
await page.getByRole('button').nth(2).click();
await page.getByRole('button').last().click();
```

nth(number) is used where the number is the index of the element on the page.

Exact Match

```
await page.getByRole('button', { name: 'Login',
exact: true }).click();
```

Locator Priority Order

1) getByRole() (Best)
2) getByLabel()
3) getByText()
4) getByTestId()
5) locator() with CSS Selectors
6) XPath (Last resort)

Locator quality determines test reliability.

Chaining Locators

```
await page.getByRole('form').getByRole('button', { name: 'Submit' }).click();
```

Filtering

```
await page.getByRole('listitem')
  .filter({ hasText: 'Product A' })
  .getByRole('button', { name: 'Add to cart' })
  .click();
```

Using Has

```
await page.locator('article', {
  has: page.getByText('Jump Rope')
}).getByRole('button', { name: 'Add' }).click();
```

Do

Prefer role-based locators.

Write locators as users perceive the UI.

Use chaining instead of long CSS selectors.

Use has and filter for dynamic content.

Don't

Use long brittle CSS chains.

Depend on auto-generated class names.

Overuse XPath.

Use index-based .nth() without reason.

Real World Patterns

Login

```
await page.getByLabel('Email').fill('');
await page.getByLabel('Password').fill('!');
await page.getByRole('button', { name: 'Login' }).click();
```

Adding an item to a cart

```
await page.getByRole('listitem')
  .filter({ hasText: 'Jump Rope' })
  .getByRole('button', { name: 'Add to cart' })
  .click();
```


Next: Waits — Because finding an element isn't enough. Timing matters.

Waits

Waiting is how you synchronize with the app — not how you “add time”. If you're using hard waits, your locator or assertion is probably wrong.

Wait for UI

page.waitForSelector()	`await page.waitForSelector('#checkout-submit');`
locator.waitFor()	`await page.locator('#cart-count').waitFor({ state: 'visible' });`

Wait for Navigation

page.waitForURL()	`await page.waitForURL('**/dashboard');`
page.waitForLoadState()	`await page.waitForLoadState('networkidle');`

Wait for Events / Debug

page.waitForTimeout()	`await page.waitForTimeout(2000);` ! Debug Only !
page.waitForEvent()	`await page.waitForEvent('popup');`

Prefer Assertions Over Waits

```
await expect(page.getByTestId('checkout-submit')).toBeVisible();
await expect(page).toHaveURL(/dashboard/);
```

Click + Navigation (race-free)

```
await Promise.all([
  page.waitForURL('**/dashboard'),
  page.getByRole('button', { name: 'Login' }).click(),
]);
```

Wait for Dynamic UI Update

```
await expect(page.getByTestId('cart-count')).toHaveText('1');
```

Wait for Popup

```
const popupPromise = page.waitForEvent('popup');
await page.getByRole('link', { name: 'Open details' }).click();
const popup = await popupPromise;
```

Wait for Element to be Ready

```
await page.locator('#checkout-submit').waitFor({ state: 'visible' });
await page.locator('#checkout-submit').click();
```

If you feel the need to add page.waitForTimeout(), you probably have a locator or synchronization problem.

Do

Prefer expect() assertions (they auto-retry).

Use waitForURL() when navigation is expected.

Use locator.waitFor() when you care about element state.

Use waitForEvent() when you trigger an event (popup/download/dialog).

Don't

Don't use waitForTimeout() in real tests.

Don't double-wait (wait then click).

Don't overuse networkidle on SPAs (it can be misleading).

Don't wait for selectors you immediately assert.

Waiting Priority Order

- expect() assertions (best)
- waitForURL()
- locator.waitFor()
- waitForSelector()
- waitForTimeout() (Debug only)

When should I wait

UI becomes visible?

- → Use expect(locator).toBeVisible()

URL changes after action?

- → Use waitForURL() or expect(page).toHaveURL()

Popup / download triggered?

- → Use waitForEvent()

Element state changes?

- → Use locator.waitFor({ state })
- Just adding time?
- → Don't. Fix your locator!

Next: Actions — Now that you can move, find, and synchronize… you can interact reliably.

Actions

Actions simulate real user interactions with the page, like Clicking, Typing, Selecting, Hovering and Dragging.

Clicking

`await page.getByRole('button', { name: 'Login' }).click();`	Click a button with a name of Login
`await page.getByRole('button', { name: 'Login' }).dblclick();`	Double click an element
`await page.getByRole('button', { name: 'Login' }).click({ button : 'right'});`	Right click an element
`await page.getByRole('button', { name: 'Login' }).click({ modifiers: ['Control'] });`	Hold control while clicking

Fill & Type and Clear

`await page.getByLabel('Email').fill('');`	Fill an input for label Email
`await page.getByLabel('Email').type('');`	Type in an input for label Email
`await page.getByLabel('Email').fill('')`	Clear the field for label Email

Press Keys

`await page.getByLabel('Search').press('Enter');`	Press the enter key on Search field
`await page.keyboard.press('Escape');`	Press Escape key Globally

Check boxes and Radios

`await page.getByLabel('Remember me').check();`	Check the checkbox for "Remember me" the Label
`await page.getByLabel('Remember me').uncheck();`	Unchecks the checkbox for "Remember me" the Label
`await page.getByLabel('Visa').check();`	Select a radio with a "Visa" Label

Dropdowns

`await page.getByLabel('Country').selectOption('ZA');`	Selects "ZA" from the "Country" dropdown
`await page.getByLabel('Tags').selectOption(['tag1', 'tag2']);`	Selects multiple options from the "Tags" dropdown

File Actions

`await page.getByLabel('Upload').setInputFiles('tests/files/sample.pdf');`
Set the File Upload file

`await page.getByLabel('Upload').setInputFiles([]);`
Unset the File Upload file

Scrolling, focus and blur

`await page.getByTestId('landing-features').scrollIntoViewIfNeeded();`

`await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));`

Focus

`await locator.focus();`

Blur

`await page.keyboard.press('Tab');`

Do

Assert something meaningful after an action like an URL, toast or heading.

Use scrollIntoViewIfNeeded() before asserting toBeInViewport() or interacting with off-screen content.

Don't

Don't overuse force: true. It bypasses safety checks.

Don't click "random text" when a role exists

Mouse Actions

`await page.getByTestId('menu-products').hover();`	Use hover for menus/tooltips that only appear on mouseover.
`await page.getByTestId('source').dragTo(page.getByTestId('target'));`	If drag/drop is custom and flaky, you can fall back to page.mouse (advanced).

Mouse Advanced

`await page.mouse.move(100, 200);`	Use only when the app requires pixel-level control.
`await page.mouse.down();`	Press and hold the mouse button
`await page.mouse.up();`	Release the mouse button

Next: Assertions — Now that we can work with element we found. We need to make sure they did what we expected.

Assertions

If you only act and never assert — you are scripting, not testing.

Visibility – UI

`await expect(page.getByText('Invalid email')).toBeVisible();`	Expects an element to be visible.
`await expect(locator).toBeHidden();`	Expects an element to be invisible / hidden.

Text - Content

`await expect(locator).toHaveText('Success');`	Expect a locator to have exact text.
`await expect(locator).toContainText('Success');`	Expect a locator to **contain** the text. (wildcard)

Value – Input Value

`await expect(locator).toHaveValue('testID1');`	Expect a locator to have a **specific** value.

URL - Page

`await expect(page).toHaveURL('/dashboard');`	Expect the URL of the page to contain text.

Title - Page

`await expect(page).toHaveTitle('Dashboard');`	Expect the Title of the page to be specific text.

Count - Collections

`await expect(locator).toHaveCount(3);`	Expect a locator count of 3 elements.

Attribute - Element

`await expect(locator).toHaveAttribute('type', 'submit');`	Expect an element to have a specific attribute.

Checked state – State

`await expect(locator).toBeChecked();`	Expect a element to be checked.
`await expect(locator).not.toBeChecked();`	Expect a element not to be checked.

Enabled / Disabled - Usability

`await expect(locator).toBeEnabled();`	Expect a element to be enabled / usable.
`await expect(locator).toBeDisabled();`	Expect an element to be disabled / unusable.

State - UI

`await expect(locator).toBeEditable();`	Expect an input element to be editable.
`await expect(locator).toBeEmpty();`	Expect an element to be empty.

To Know

Playwright assertions do the following:

- Auto-waits
- Retry until a time is reached
- Reduces flakiness
- Has async awareness
- Fails with meaningful errors

It does not act as a snapshot check.

Don't

`expect(await locator.isVisible()).toBe(true);` This is snapshot like

Do

`await expect(locator).toBeVisible();`

- Always use await expect().
- Assert behavior.
- Assert user visible outcomes.
- Prefer semantic locators / page objects (see locator priority p.3).
- Use toContainText() for flexibility.
- Use “not” carefully

Example: `await expect(loginPage.authError).toContainText('Invalid');`

DOM

`await expect(locator).toBeAttached();` Expect element to exist in the DOM

Soft Assertions

`await expect.soft(locator).toBeVisible();`	Useful when you want multiple validations before failing, you want a summary of failures.

Regex Assertions

`await expect(title).toHaveText(/Order \d+/);`	Useful for dynamic order numbers, timestamp, version numbers, emails, URLs and text with structure.

Next: Visual testing, because it is another form of assertions.

Visual Testing

Visual tests verify what the user sees, not just what the application returns.

Screenshot Assertions

expect(page).toHaveScreenshot();	Compare a screenshot of the page with a baseline image.
expect(locator).toHaveScreenshot();	Compare a screenshot of a specific element.
page.screenshot({ fullPage: true });	Capture a screenshot of the entire page.
expect(page).toHaveScreenshot({ mask: [locator] });	Mask areas that change often.
locator.screenshot();	Capture a screenshot of a specific element.

Example:

```
await page.goto('/dashboard');
await expect(page).toHaveScreenshot();
```

Visual testing on an element

```
await expect(page.locator('#product-card'))
  .toHaveScreenshot();
```

Ignoring small pixel differences

```
expect(page).toHaveScreenshot({ maxDiffPixels: number });
```

Example:

```
await expect(page).toHaveScreenshot({
  maxDiffPixels: 100
});
```

Note:

When running a visual test for the first time, Playwright automatically captures and saves the screenshot as the baseline snapshot. This baseline image becomes the reference used for future comparisons. On subsequent test runs, Playwright takes a new screenshot and compares it against the saved baseline. If visual differences are detected beyond the allowed threshold, the test fails and a diff image is generated to highlight the changes.

To Know

Keep screenshots stable.
Don't include stuff like timestamps, animation or rotating banners.
Test components instead of full pages.
Run visual tests in a consistent environment.

Don't

Don't visual test dynamic content.
Don't mix visual and functional failures.
Don't ignore screenshot diffs.

Do

Use visual testing for UI regressions.
Compare small components when possible.
Store baseline images in the repo.

Next: File Uploads & Dialogs — Because some actions trigger browser dialogs or file selectors.

File Uploads & Dialogs

Uploading files and working with dialogs does not have to be difficult.

Input Files

locator.setInputFiles();	Upload a file using an <input type="file"> element.
locator.setInputFiles([]);	Clear a previously selected file.
page.waitForEvent('filechooser');	Wait for the browser file chooser dialog to open.
fileChooser.setFiles();	Select files from the file chooser dialog.
page.on('dialog');	Listen for browser dialogs (alert, confirm, prompt).

Dialogs

dialog.accept();	Accept a dialog (OK / Confirm).
dialog.dismiss();	Dismiss a dialog (Cancel).

PDF

page.pdf();	Generate a PDF of the current page (Chromium only).

How To:

Set File uploads:
```
await page.locator('#upload').setInputFiles('tests/files/sample.pdf');
```

Choose Files:
```
const fileChooser = await page.waitForEvent('filechooser');
await fileChooser.setFiles('tests/files/image.png');
```

Accept dialogs:
```
page.on('dialog', dialog => dialog.accept());
```

Dismiss dialogs:
```
dialog.dismiss();
```

Generate a PDF:
```
await page.pdf({ path: 'page.pdf' });
```

Drag and Drop example:
```
await page.locator('#dropzone input[type="file"]')
  .setInputFiles('tests/files/upload.pdf');
```

To Know

Playwright must be listening for dialogs before they appear.
Direct file uploads are faster and more reliable than interacting with the OS file picker.
Playwright cannot control the native operating system file picker.
Playwright uploads files directly through the DOM, which is faster and more reliable.

Don't

Ignore dialogs.
Upload files from absolute paths.
Don't interact with the OS file picker.
Don't simulate drag-and-drop for file uploads

Do

Use test files in your repository.
Accept or dismiss dialogs.
Use setInputFiles for uploads.
Listen for dialogs before triggering them.
Use relative file paths.
Validate upload results.

Next: Network — Because not all behavior happens in the UI.

Network

Knowing what happens on the UI is good. Knowing what happens on the network is even better.

Pages

page.on('request');	Listen for outgoing network requests.
page.on('response');	Listen for incoming network responses.
page.waitForRequest();	Wait for a specific network request.
page.waitForResponse();	Wait for a specific network response.
response.json();	Get the JSON body of a response.
response.status();	Get the HTTP status code of a response.

Listen for an event

```
page.on('request', request => {
  console.log(request.url());
});
```

Wait for a request

```
const response = await page.waitForResponse('**/api/orders');
```

Inspect a response

```
const data = await response.json();
expect(response.status()).toBe(200);
```

Example workflow:

```
const response = await page.waitForResponse('**/api/cart');
await page.click('#add-to-cart');
expect(response.status()).toBe(200);
```

To Know

Modern web applications rely heavily on APIs.
Monitoring network traffic helps verify backend communication and diagnose test failures.

Don't

Don't rely only on UI changes.
Don't listen to every request unnecessarily.
Don't assume API requests finish instantly

Do

Wait for network responses when validating async actions.
Filter requests using URL patterns.
Validate response data when possible.
Monitor important API calls.
Wait for requests triggered by actions.
Validate status codes.

Next: API Testing — Because many application features rely on backend APIs.

API Testing

Many actions happen through API, Gotta test them all.

API Requests

```
const requestContext = await request.newContext();
```

request.get(url);	Send a **GET** request to an API endpoint.
request.post(url, data);	Send a **POST** request with data to an API endpoint.
request.put(url, data);	Replace data on an API endpoint using **PUT**.
request.delete(url, data);	Send a **DELETE** request to remove data from an API endpoint.
request.patch(url, data);	Update part of the data on an API endpoint.

Example:

```
const response = await request.post('/api/login', {
  data: { email: 'user@example.com', password: 'pass123' }
});
```

Reading the responses

response.status();	Get the HTTP status code from the response.
response.json();	Get the JSON body from the response.
response.ok();	Check if the response was successful.

Example:

```
expect(response.status()).toBe(200);
const body = await response.json();
expect(body.token).toBeTruthy();
```

API Routing / Mocking

page.route(url, handler);	Intercept and handle matching network requests.
page.unroute(url);	Stop intercepting matching network requests.

Example:

```
await page.route('**/api/products', async route => {
  await route.fulfill({
    status: 200,
    body: JSON.stringify([{ id: 1, name: 'Mock Product' }])
  });
});
```

To Know

Use API requests for fast backend validation.
Validate both status and response body.
Use API setup to speed up UI tests.
Over-mocking can hide real integration problems.
API requests → test the service
Route mocking → control the service

Don't

Don't mix UI validation and API validation without purpose.
Don't assume success from status code alone.
Don't mock everything.
Don't leave route handlers active accidentally.

Do

Use API tests for backend-heavy features.
Keep test data predictable.
Use mocking when backend access is unstable.
Clean up route handlers when no longer needed

Next: Evaluation — Because automation can execute JavaScript directly in the browser.

Evaluation

If you only act and never assert — you are scripting, not testing.

Running JavaScript in the Browser

page.evaluate(fn);	Execute JavaScript inside the browser page.
locator.evaluate(fn);	Execute JavaScript on a specific element.

Example

```
await page.evaluate(() => document.title);
```

Reading DOM Content

locator.innerHTML();	Get the HTML content inside an element.
locator.innerText();	Get the visible text of an element.
locator.textContent();	Get the raw text content of an element.
locator.getAttribute(name);	Get the value of a specific element attribute.

Example

```
const text = await page.locator('#message').innerText();
```

Example Workflow:

```
await page.goto('/dashboard');
const title = await page.evaluate(() => document.title);
expect(title).toContain('Dashboard');
```

To Know

Prefer locators over evaluate() when possible.
Keep evaluation scripts simple,
evaluate() runs JavaScript inside the browser, while locator methods safely retrieve information from elements.

Don't

Don't overuse evaluate().
Don't manipulate application state unnecessarily.
Don't assume browser variables exist

Do

Use evaluate() for browser-side logic.
Use locator helpers for reading content.
Validate attributes when testing UI behavior

You now have the core building blocks for reliable Playwright automation.